FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA – CENTRO DE CIÊNCIAS TECNOLÓGICAS

Lista de Exercícios – Unidade II

Algoritmos de Pesquisa e de Ordenação

Disciplina: Construção e Análise de Algoritmos (T198 – Turma 9)
Prof. Caio Marques Silva | Semestre 2026.2

Aluno(a): _____ **Matrícula:** _____

Orientações. Esta lista cobre o item 02.01 do Plano de Ensino: pesquisa linear, pesquisa binária, ordenação por seleção, por trocas, por inserção, por intercalação, *heapsort*, *quicksort* e ordenação por contagem. São 50 questões organizadas em quatro níveis. Em todas as questões de análise, justifique a resposta: a complexidade afirmada sem justificativa não pontua. Salvo indicação em contrário, considere vetores de n elementos, indexados a partir de zero, e ordenação crescente.

Nível I – Questões 1 a 15: conceitos fundamentais

Conceituais, sobre o funcionamento das estruturas e dos algoritmos.

Questão 1. Descreva o funcionamento da **pesquisa linear**. Ela exige que o vetor esteja ordenado? Justifique.

Questão 2. Qual é o **pré-requisito obrigatório** da pesquisa binária? O que acontece se ele for violado?

Questão 3. Defina **estabilidade** de um algoritmo de ordenação. Cite um algoritmo estável e um não estável entre os estudados.

Questão 4. Defina ordenação ***in place***. Quais dos algoritmos estudados são *in place* e quais não são?

Questão 5. Explique uma passagem da **ordenação por seleção**. Ao final da execução, quantas trocas o algoritmo terá realizado?

Questão 6. Por que a ordenação por trocas é chamada de *Bubble Sort*? O que se pode afirmar sobre o vetor ao final da primeira passagem?

Questão 7. Explique a ordenação por **inserção** usando a analogia das cartas de baralho.

Questão 8. Quais são as duas fases do **mergesort**? O que faz exatamente a rotina `merge`?

Questão 9. No **quicksort**, o que é o **pivô** e o que faz a rotina de **partição**?

Questão 10. Defina **max-heap**. Dado um nó no índice i de um heap representado em vetor, quais são os índices dos filhos e do pai?

Questão 11. Por que a ordenação por **contagem** não compara elementos entre si? Que restrição isso impõe sobre os dados de entrada?

Questão 12. Simule a **ordenação por seleção** sobre $v = [\,7,\ 3,\ 9,\ 1\,]$, mostrando o vetor ao final de cada passagem.

Questão 13. Simule a **ordenação por inserção** sobre $v = [\,4,\ 2,\ 8,\ 1\,]$, mostrando o vetor após cada inserção.

Questão 14. Em um vetor ordenado com 1024 elementos, quantas comparações a pesquisa binária faz no pior caso? Compare com a pesquisa linear.

Questão 15. Diferencie algoritmos de ordenação **por comparação** dos algoritmos **por distribuição**. Classifique os nove algoritmos estudados.

Nível II – Questões 16 a 30: conceitos e análise assintótica
Complexidade nos casos melhor, médio e pior, com justificativa.

Questão 16. Complete a tabela abaixo para todos os algoritmos de ordenação estudados, indicando melhor caso, caso médio, pior caso, espaço auxiliar e estabilidade.

Questão 17. Por que a ordenação por seleção é $\Theta(n^2)$ **inclusive no melhor caso**, enquanto a inserção chega a $\Theta(n)$? O que isso revela sobre sensibilidade à entrada?

Questão 18. Qual entrada leva a ordenação por inserção ao pior caso? E ao melhor? Justifique com o número de deslocamentos.

Questão 19. Explique a otimização do *bubble sort* com bandeira de parada antecipada. Qual passa a ser o melhor caso e por quê?

Questão 20. Escreva a relação de recorrência do mergesort e resolva-a pelo Teorema Mestre, identificando a, b e d.

Questão 21. Escreva a recorrência da pesquisa binária e resolva-a pelo Teorema Mestre. Por que $a = 1$?

Questão 22. Em que situação o quicksort atinge o pior caso $\Theta(n^2)$? Por que a escolha aleatória do pivô (ou a mediana de três) atenua o problema?

Questão 23. Escreva as recorrências do quicksort no melhor e no pior caso e resolva ambas.

Questão 24. Por que o `heapify` custa $\Theta(\log n)$? E por que a construção do heap custa $\Theta(n)$, e não $\Theta(n \log n)$?

Questão 25. A ordenação por contagem é $\Theta(n+k)$. Para que valores de k ela deixa de ser vantajosa frente ao mergesort? Dê um exemplo numérico.

Questão 26. Compare o consumo de memória auxiliar de mergesort, quicksort e heapsort, explicando a origem de cada custo.

Questão 27. Por que a ordenação por contagem percorre o vetor de entrada **da direita para a esquerda** na fase de construção da saída? O que aconteceria se o percurso fosse invertido?

Questão 28. Por que a pesquisa binária não traz ganho quando aplicada a uma **lista encadeada**?

Questão 29. Deduza formalmente o caso médio da pesquisa linear **com sucesso**, supondo que a chave esteja em qualquer posição com igual probabilidade.

Questão 30. Considere um mergesort híbrido que, quando o subvetor atinge tamanho $n \leq k$, passa a usar ordenação por inserção. Analise a complexidade resultante em função de n e k e explique por que a técnica é usada na prática.

Nível III – Questões 31 a 45: aprofundamento
Demonstrações, variantes dos algoritmos e análises mais finas.

Questão 31. Demonstre que todo algoritmo de ordenação **baseado em comparações** executa $\Omega(n \log n)$ comparações no pior caso.

Questão 32. Por que a ordenação por contagem, sendo $\Theta(n)$ quando $k = O(n)$, não contradiz o limite inferior da questão anterior?

Questão 33. Deduza o caso médio do quicksort a partir da recorrência $T(n) = \frac{1}{n}\sum_{i=1}^{n}\big[T(i-1) + T(n-i)\big] + cn$.

Questão 34. Prove que a construção de um max-heap a partir de um vetor arbitrário custa $\Theta(n)$.

Questão 35. Suponha que o quicksort produza **sempre** partições na proporção 90%–10%. Mostre que a complexidade continua $\Theta(n \log n)$.

Questão 36. Um vetor contém muitos elementos **repetidos** (por exemplo, todos iguais). Mostre que o quicksort tradicional degenera e proponha uma variante que resolva o problema.

Questão 37. Mostre como transformar qualquer algoritmo de ordenação **não estável** em estável. Qual é o custo adicional dessa transformação?

Questão 38. Descreva o **mergesort bottom-up** (iterativo) e analise sua complexidade. Que vantagem ele tem sobre a versão recursiva?

Questão 39. Para encontrar o k-ésimo menor elemento de um vetor **não ordenado**, compare ordenar o vetor com usar o *quickselect*. Escreva e resolva a recorrência do quickselect no caso médio.

Questão 40. Um vetor ordenado foi **rotacionado** em uma posição desconhecida (por exemplo, $[7, 9, 12, 1, 3, 5]$). Adapte a pesquisa binária para localizar uma chave nesse vetor em $\Theta(\log n)$.

Questão 41. O *radix sort* ordena n números de d dígitos na base b usando a ordenação por contagem em cada dígito. Justifique a complexidade $\Theta(d(n + b))$ e discuta a escolha de b.

Questão 42. Seja I o número de **inversões** de um vetor (pares (i, j) com $i < j$ e $v[i] > v[j]$). Prove que a ordenação por inserção executa $\Theta(n + I)$ operações.

Questão 43. Explique como ordenar um arquivo de registros que **não cabe na memória principal** (ordenação externa). Qual algoritmo é a base da solução e por quê?

Questão 44. Mostre, com um contraexemplo concreto, que o **heapsort não é estável**.

Questão 45. Enuncie a **invariante de laço** da ordenação por inserção e use-a para provar a correção do algoritmo (inicialização, manutenção e término).

Nível IV – Questões 46 a 50: escolha e implementação

Em cada cenário, escolha o algoritmo mais adequado, **justifique a escolha comparando-a com ao menos uma alternativa descartada** e implemente a solução na linguagem de sua preferência, informando a complexidade de tempo e de espaço.

Questão 46. Boletim acadêmico. A secretaria precisa ordenar os registros de 50 000 alunos pela nota final, um número **inteiro de 0 a 100**. Alunos com a mesma nota devem permanecer na ordem alfabética em que já chegaram da listagem anterior. Escolha, justifique e implemente.

Questão 47. Telemetria de sensores. Um sistema recebe a cada minuto um vetor de 10 000 leituras **quase ordenado** por *timestamp*: apenas algumas dezenas de leituras chegam fora de ordem, por atraso de rede. Escolha, justifique e implemente.

Questão 48. Sistema embarcado de tempo real. Um controlador com memória muito restrita deve ordenar até 5 000 medições e precisa de **garantia de tempo no pior caso** para respeitar o prazo do ciclo de controle. Escolha, justifique e implemente.

Questão 49. Processamento de log. Um arquivo de 200 GB de registros deve ser ordenado por identificador, em uma máquina com 8 GB de RAM. Escolha, justifique e descreva a implementação (não é necessário código completo).

Questão 50. Consulta a base estática. Um serviço mantém uma base **estática** de 10 milhões de identificadores e responde a cerca de $m = 100\,000$ consultas de existência por dia. Compare a estratégia de busca linear a cada consulta com a de ordenar uma única vez e usar pesquisa binária. A partir de quantas consultas a segunda compensa? Implemente a solução escolhida.

"The ultimate test of your knowledge is your capacity to convey it to another." — Richard Feynman